

Do you plan on coming to
the MP2 party today after class
in Siebel 4124?

CS 340

Central Processing Unit

clicker.cs.illinois.edu

Q1

~Code~
340



Updates

HW3 MP1...

20 point MC, HW

30 points coding

1. HW 4 - easy

1. MP 2 - out MP party

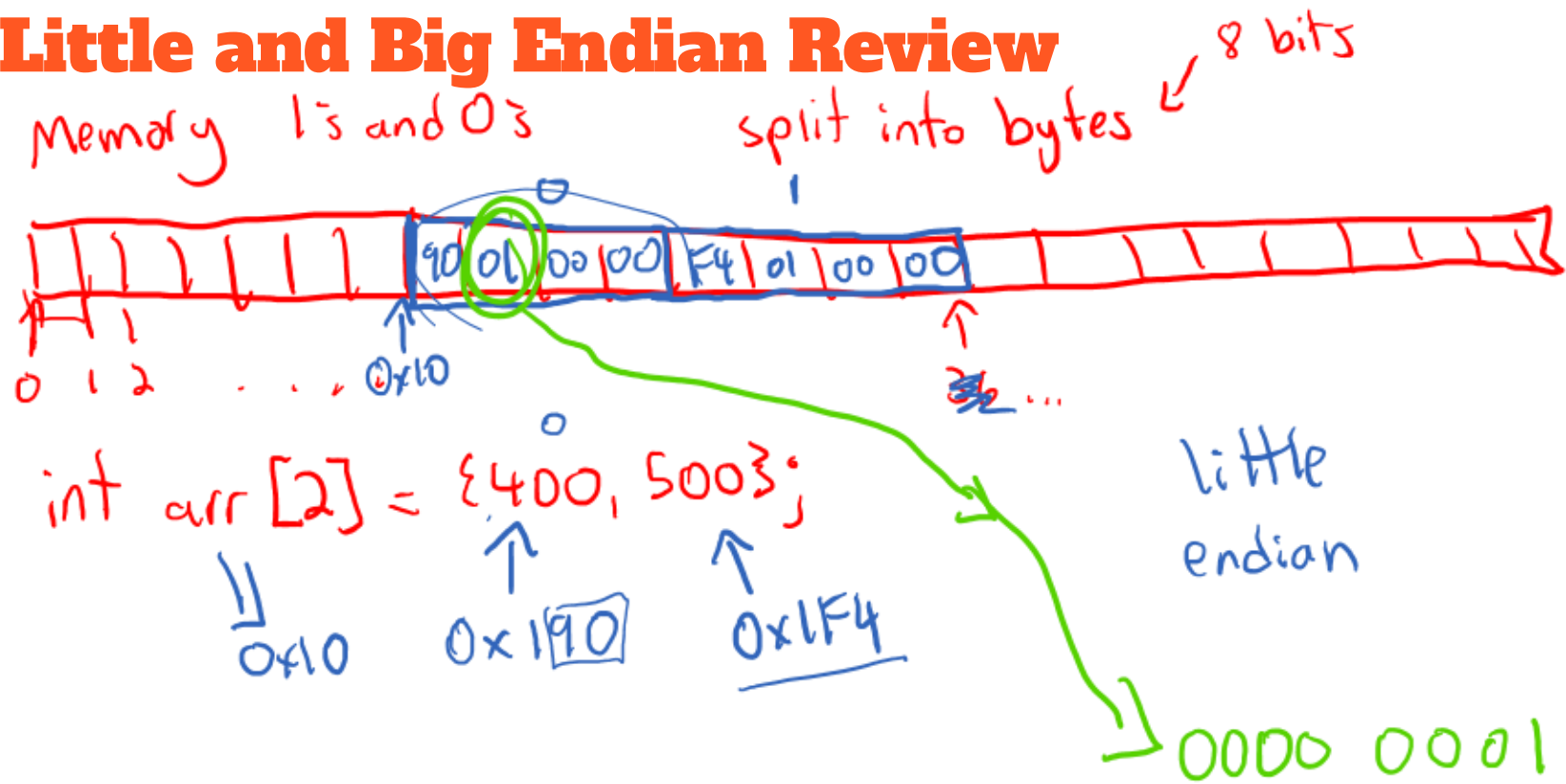
1. Canvas grades ←

1. Exam 1 ← next week
Tues - review
Thurs - no class

study guide
on website

practice exam

Little and Big Endian Review



Endianness and Memory Layout

[View...](#)

Below we show ten bytes of **little-endian memory** at several addresses, using 2-hex-digit representations of each byte.

Address	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009
Value	F6	DC	6D	CD	58	AF	22	49	BC	E3

Suppose a `uint16_t *p` (i.e. a pointer to unsigned 16-bit integers) has value $p = 1006$.

What is the value of `p[1]`? Answer in hexadecimal.

`p[1]`

integer in base 16

E3BC

Save & Grade

Save only

$$p[1] = *(p+1) \\ 1006+2 = 1008$$

clicker.cs.illinois.edu

Q2

~Code~
340



Endianness and Memory Layout

[View...](#)

Below we show ten bytes of big-endian memory at several addresses, using 2-hex-digit representations of each byte.

Address	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009
Value	D5	14	F2	07	B3	4E	3A	C4	BD	2C

Suppose a `uint16_t *p` (i.e. a pointer to unsigned 16-bit integers) has value $p = 1006$.

What is the value of $*(p - 1)$? Answer in hexadecimal. 2-bytes

`*(p - 1)` integer in base 16

B34E

Save & Grade

Save only

$$p-1 \\ 1006-2 = 1004$$

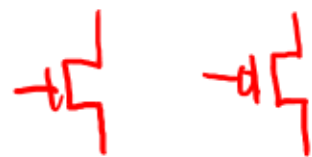
clicker.cs.illinois.edu

Q3

~Code~
340



Review



math
selection
logic

storage
hardware

fast - we don't have a lot
slow - we have a lot

caching

— CPU



binary — write in hex
easier to
read

Central Processing Unit CPU

Today's LGs - Have a high level understanding of how we get from voltages across hardware to C.

- Be able to define assembly, ISA, CPU
- Be able to list the hardware components in a CPU

High level logical steps → voltages across gates

C program

↓
assembly
machine code ← CPU runs

Agenda

1. CPU Hardware

2. Assembly

3. Machine Code

4. Optimization

Central Processing Unit



Hardware that reads and executes instructions.

moving voltages into different storage and logic gates

Fetch - get the next instruction from memory

Decode - what is the instruction asking

execute - does instruction

- go to next instruction

Hardware in a CPU



arithmetic logic unit

- takes voltages and performs computational operations with gates

Control unit

- gates/logic that dictate where things go

- math
- selection
- logic
- comparisons

Hardware in a CPU



general registers

Size: 8 bytes

amount: 16-32

program counter

Size: 8 bytes

Keep address of
the next instruction

instruction reg.

- 8 bytes

- current instruction

What is fed into the ALU from the Registers?

clicker.cs.illinois.edu

Q4

~Code~
340



- A) Transistors 3%
- B) The cache 70%
- C) Voltages at 16 different values 31%
- ☒ D) Voltages at 2 different values 54%

What hardware does a CPU contain?

clicker.cs.illinois.edu

Q5

~Code~
340



- ☒ A) Gates
- ☒ B) Transistors
- ~~C) RAM~~ ← memory
- ☒ D) Registers
- ~~E) SSD~~
↳ disk

What can a CPU execute?

```
5  ∨ int main(){  
6      FILE* fl = fopen("words_alpha.txt", "r");  
7      int wrd_lst_len = 0;  
8      int wrd_lst_cap = 99999;  
9      char* wrd_lst[wrd_lst_cap];  
10     int len_wrd = 0;  
11  
12  ∨ while(1){  
13      → char buf;  
14          int ret = fread(&buf, 1, 1, fl);  
15          if(ret != 1) break;  
16  
17          // if end of the line  
18  ∨      if(buf == '\n'){  
19          //if length of word is  
20  ∨          if(len_wrd == 6){
```

— what registers to put
what

— what to put in ALU
when

....

cannot run C code
directly

What does a CPU run?

c code

```
5 int main(){
6     FILE* fl = fopen("words_alpha.txt", "r");
7     int wrd_lst_len = 0;
8     int wrd_lst_cap = 99999;
9     char* wrd_lst[wrd_lst_cap];
10    int len_wrd = 0;
11
12    while(1){
13        char buf;
14        int ret = fread(&buf, 1, 1, fl);
15        if(ret != 1) break;
16
17        // if end of the line
18        if(buf == '\n'){
19            //if length of word is
20            if(len_wrd == 6){
```

human
readable

compiler

assembly

64	mov x22, x1
65	mov x23, 0
66	lsr x1, x22, 58
67	lsl x27, x23, 6
68	mov x2, x27
69	add x1, x1, x2
70	mov x27, x1

human
readable
machine code

machine code

21	00 00 CB 22 FC 7F D3 41 0C 81 8B 21 FC 41 93
C1	00 00 B4 E2 00 00 F0 42 F0 47 F9 62 00 00 B4
F0	03 02 AA 00 02 1F D6 C0 03 5F D6 FD 7B BE A9
FD	03 00 91 F3 0B 00 F9 13 01 00 90 60 62 41 39
80	00 00 37 DF FF FF 97 20 00 80 52 60 62 01 39
F3	0B 40 F9 FD 7B C2 A8 C0 03 5F D6 1F 20 03 D5
E4	FF FF 17 FD 7B B2 A9 FD 03 00 91 F3 53 01 A9
F5	5B 02 A9 F7 63 03 A9 F9 6B 04 A9 FB 2B 00 F9
E0	03 00 91 F3 03 00 AA 00 00 00 90 01 00 30 91
00	00 00 90 00 20 30 91 92 FF FF 97 A0 57 00 F9
BF	DF 00 B9 E0 D3 90 52 20 00 A0 72 A0 A7 00 B9
A0	A7 40 B9 01 7C 40 93 21 04 00 D1 A1 4F 00 F9

cpu
needs

Assembly

General form

operation operand1 operand2

vary per ISA

↑
instruction
set
architecture

Operations

- move data ✓ load
 - store into memory from register
- Arithmetic
 - add, sub
- ~~Example~~ Compare
- Control Flow
 - Jump

Assembly Example

ARM64

→ ISA

registers

→ copy x1 data into x22

64	<u>mov x22, x1</u>
65	mov x23, 0
66	lsr x1, x22, 58
67	lsl x27, x23, 6
68	mov x2, x27
69	add x1, x1, x2
70	mov x27, x1

→ $x1 = x1 + x2$
← registers

int x
= y + x;

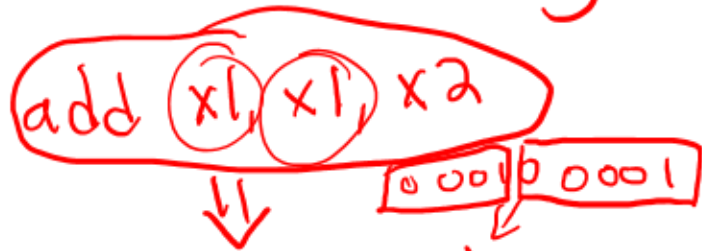
x86-64

mov %rdi, %r14

add %rsi, %rdi ← $rdi = rdi + rsi$

Machine Code

encoded assembly ISA



0x8B020021

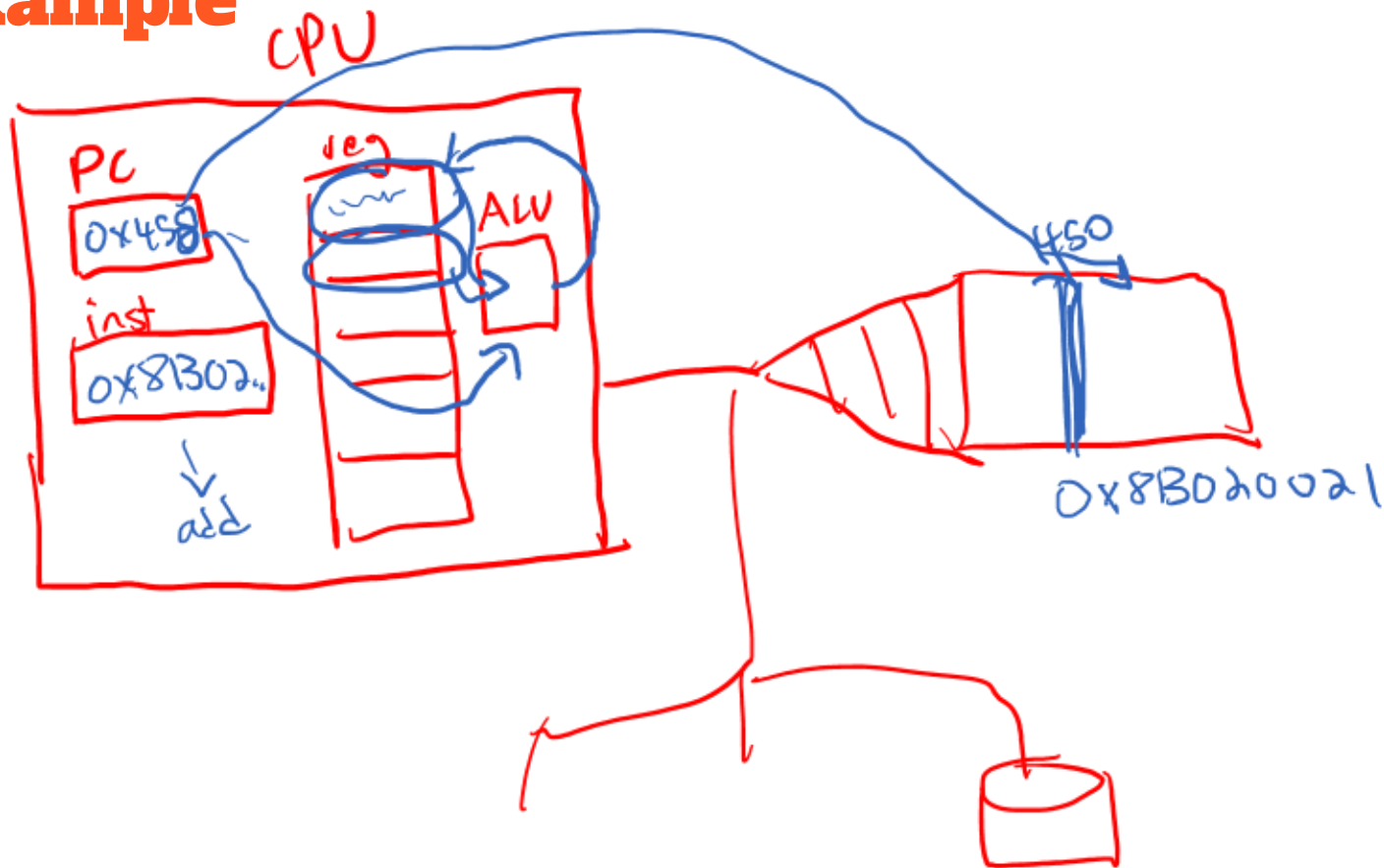
³¹↓ ¹⁰↓
1000 1011 0000 0010 ...

30th - 0 for add

9-5 - source reg

4-0 - dest

Example



Big Picture

compiler is machine code that load ascii from main.c
into cpu, does logic to turn that ascii into machine instructions
to save in memory as a.out

main.c

```
5 int main(){
6     FILE* fl = fopen("words_alpha.txt", "r");
7     int wrd_lst_len = 0;
8     int wrd_lst_cap = 99999;
9     char* wrd_lst[wrd_lst_cap];
10    int len_wrd = 0;
11
12    while(1){
13        char buf;
14        int ret = fread(&buf, 1, 1, fl);
15        if(ret != 1) break;
16
17        // if end of the line
18        if(buf == '\n'){
19            //if length of word is
20            if(len_wrd == 6){
```

gcc main.c
↑

a.out

21	00	00	CB	22	FC	7F	D3	41	0C	81	8B	21	FC	41	93
C1	00	00	B4	E2	00	00	F0	42	F0	47	F9	62	00	00	B4
F0	03	02	AA	00	02	1F	D6	C0	03	5F	D6	FD	7B	BE	A9
FD	03	00	91	F3	0B	00	F9	13	01	00	90	60	62	41	39
80	00	00	37	DF	FF	FF	97	20	00	80	52	60	62	01	39
F3	0B	40	F9	FD	7B	C2	A8	C0	03	5F	D6	1F	20	03	D5
E4	FF	FF	17	FD	7B	B2	A9	FD	03	00	91	F3	53	01	A9
F5	5B	02	A9	F7	63	03	A9	F9	6B	04	A9	FB	2B	00	F9
E0	03	00	91	F3	03	00	AA	00	00	00	90	01	00	30	91
00	00	00	90	00	20	30	91	92	FF	FF	97	A0	57	00	F9
BF	DF	00	B9	E0	D3	90	52	20	00	A0	72	A0	A7	00	B9
A0	A7	40	B9	01	7C	40	93	21	04	00	D1	A1	4F	00	F9

I am guaranteed to be able to run a C executable I made on my computer on your computer.

↑
different ISA

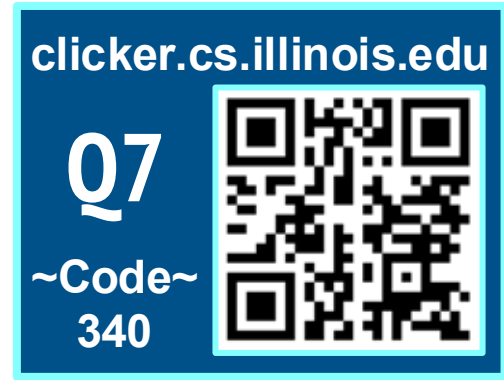


A) True

B) False

The process goes,
C code → **XXXXXX** → **machine**
code

binary → binary → binary



- A) Hexadecimal
- B) Binary
- C) Decimal
- D) Assembly

I can view bytes in which of the following formats...

clicker.cs.illinois.edu

Q8

~Code~
340



A) Hexadecimal

B) Binary

C) Decimal

~~D) Ascii~~

**I can view assembly in
which of the following
formats...**



- ☒ A) Hexadecimal
 - ☒ B) Binary
 - ☒ C) Decimal
 - ☒ D) Ascii
- ↑

**If a C program is 200 lines,
the related assembly will
be guaranteed to also be
200 lines.**



- A) True
- B) False

Optimizations

The compiler can do more or less work

-O0
-O1
-O2
-O3



more compiler work - faster executable
less compiler work - faster compiling